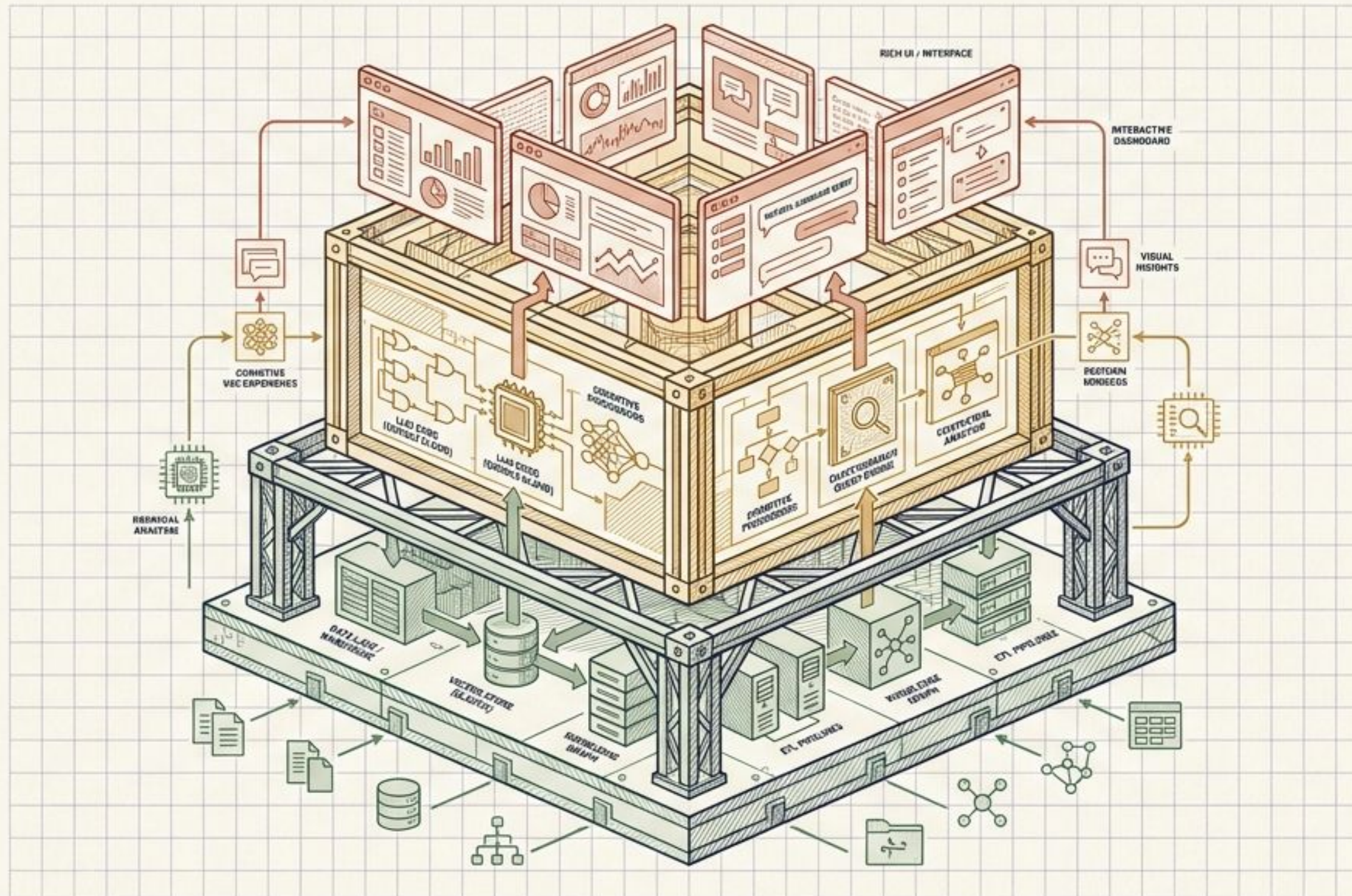
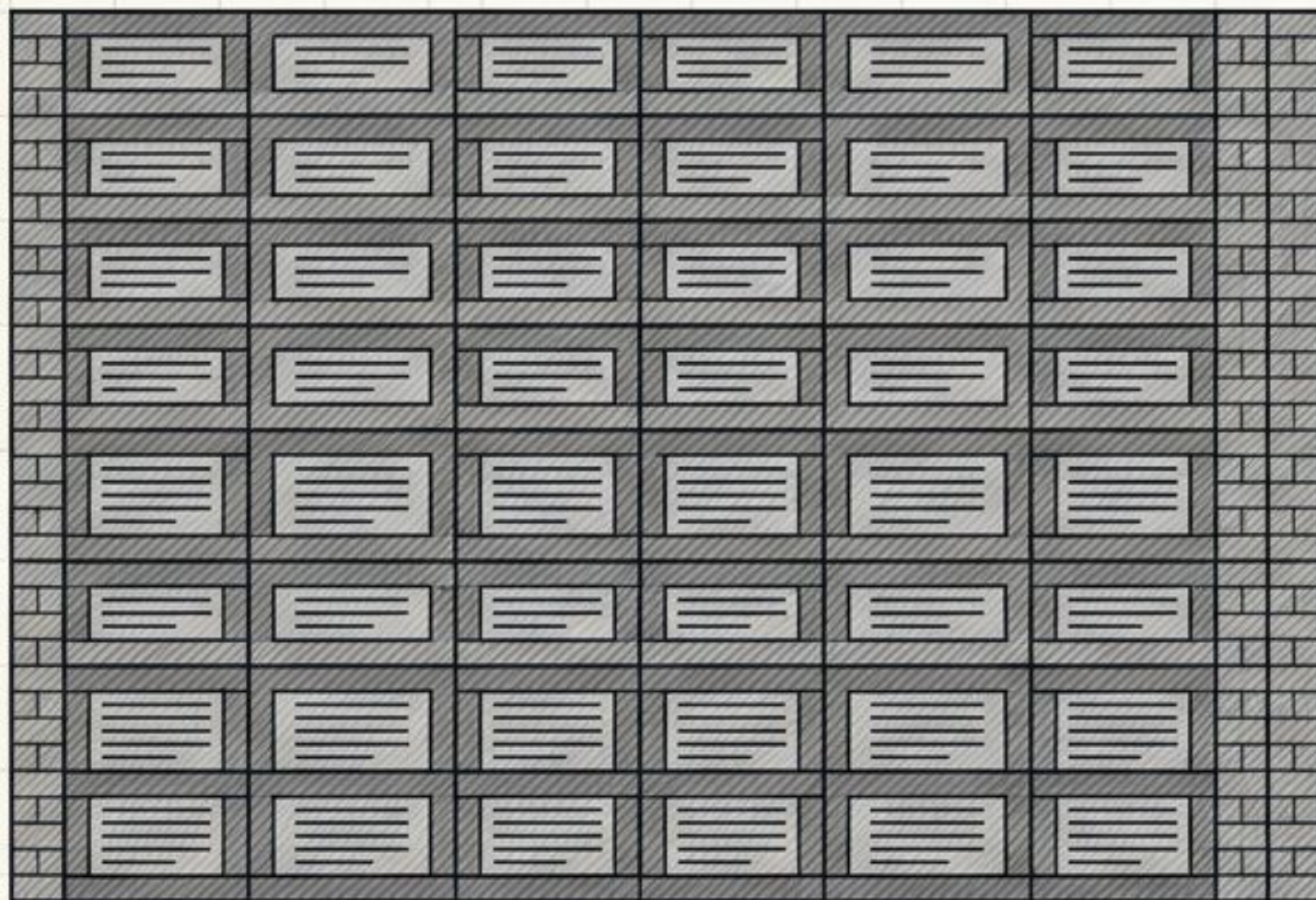


次世代AIエージェントの最適解

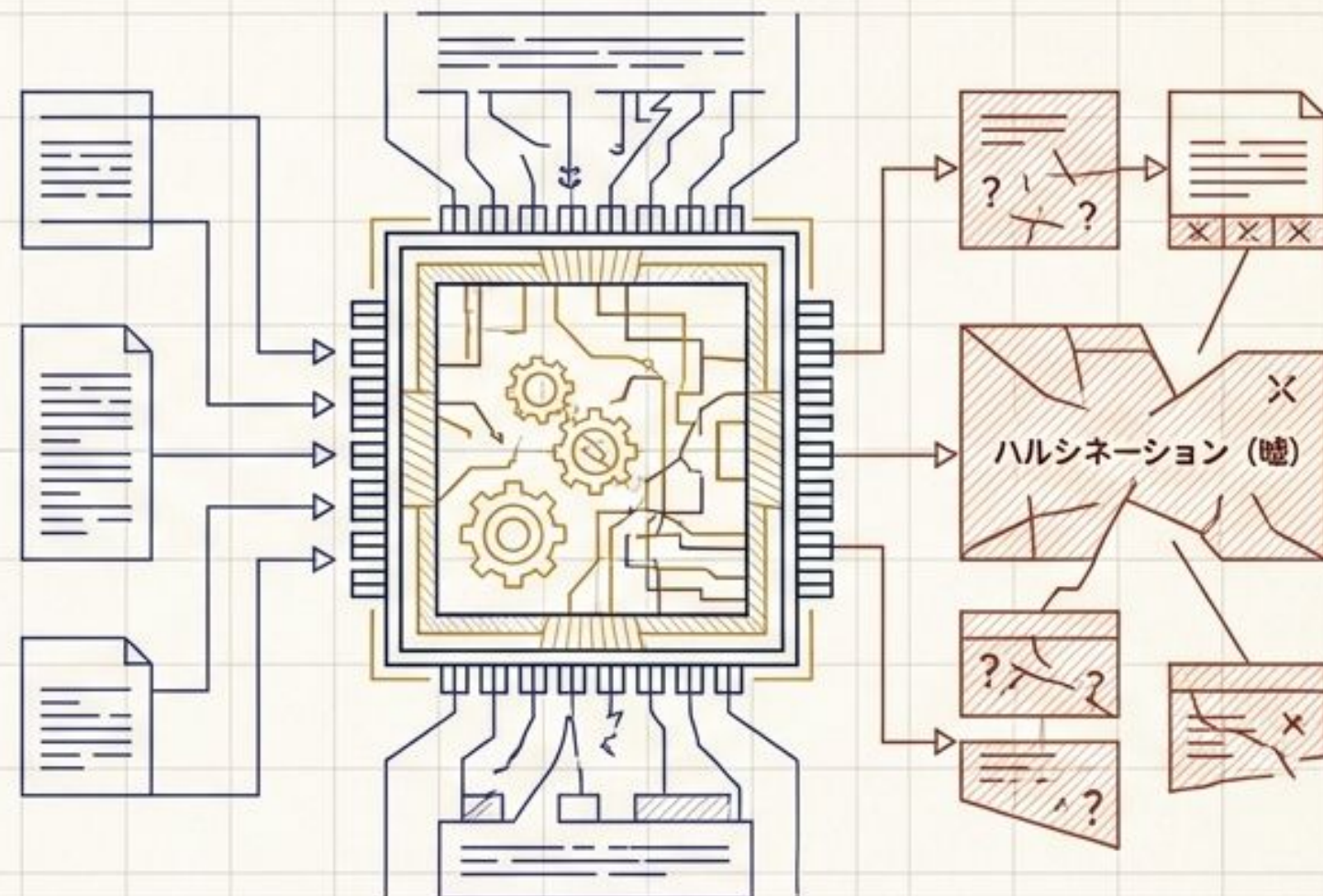
Google Cloud × Elasticで構築する「圧倒的精度」と「リッチUI」の実現



単調なテキストUI



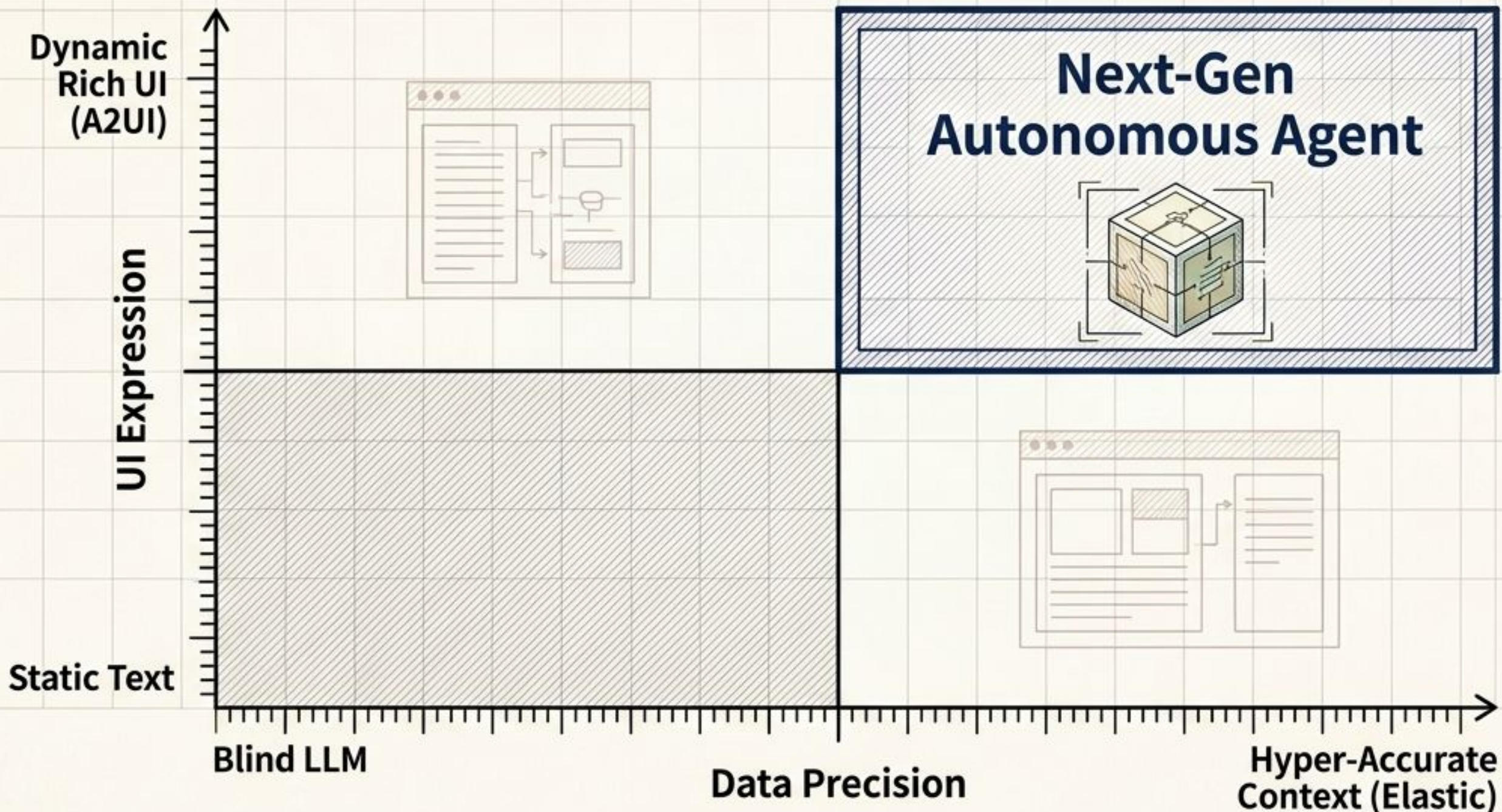
低い検索精度によるハルシネーション



単調なテキストのやり取りと、低い検索精度によるハルシネーション（嘘）が、実用的なユーザー体験（UX）を決定的に阻害している。

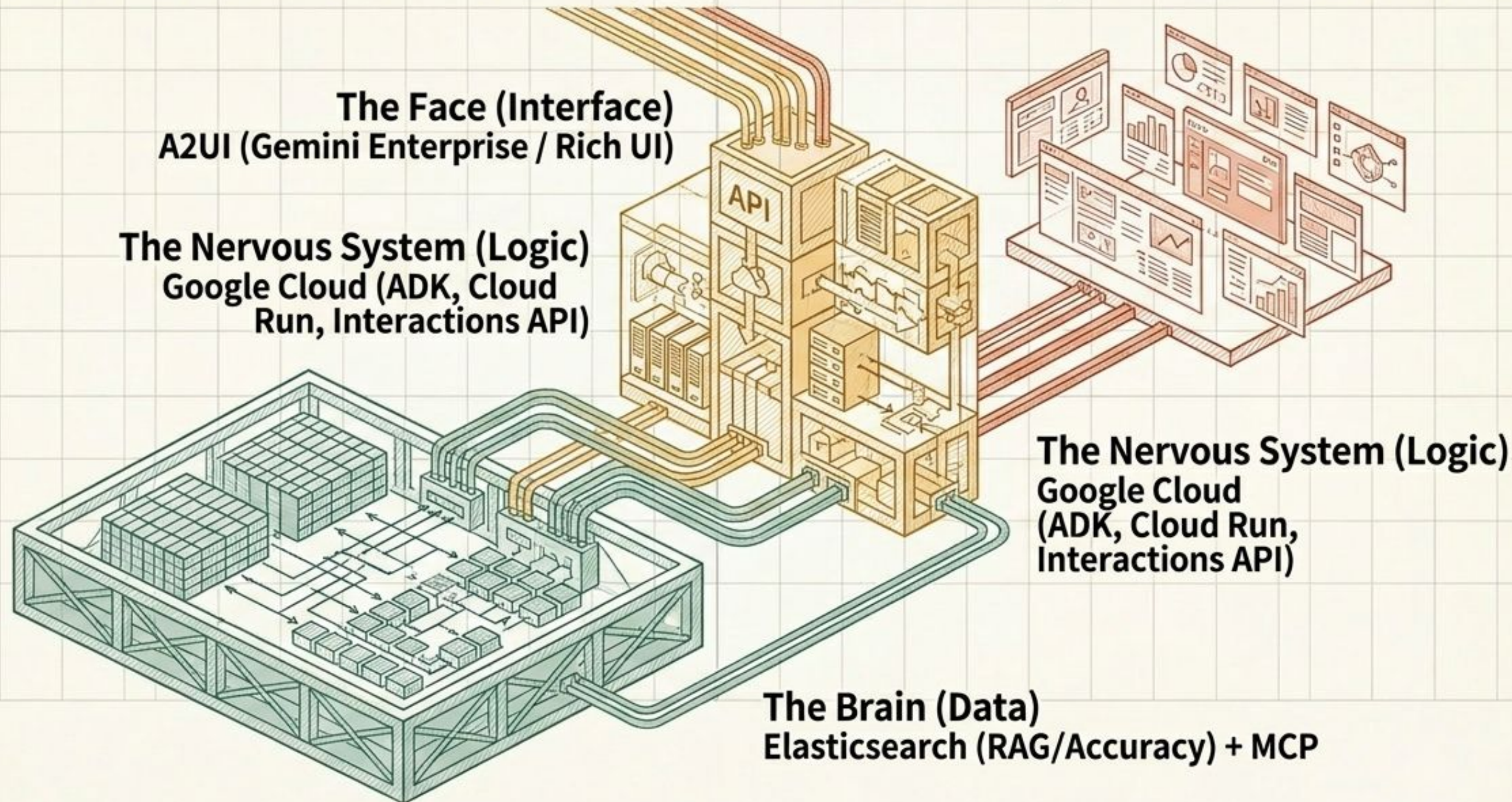
次世代エージェント：自律的な「思考」と「表現」の獲得

圧倒的な体験を生み出すための2つの鍵は、裏側の「極めて精度の高いコンテキスト」と
表側の「動的でリッチなUI表現力」である。



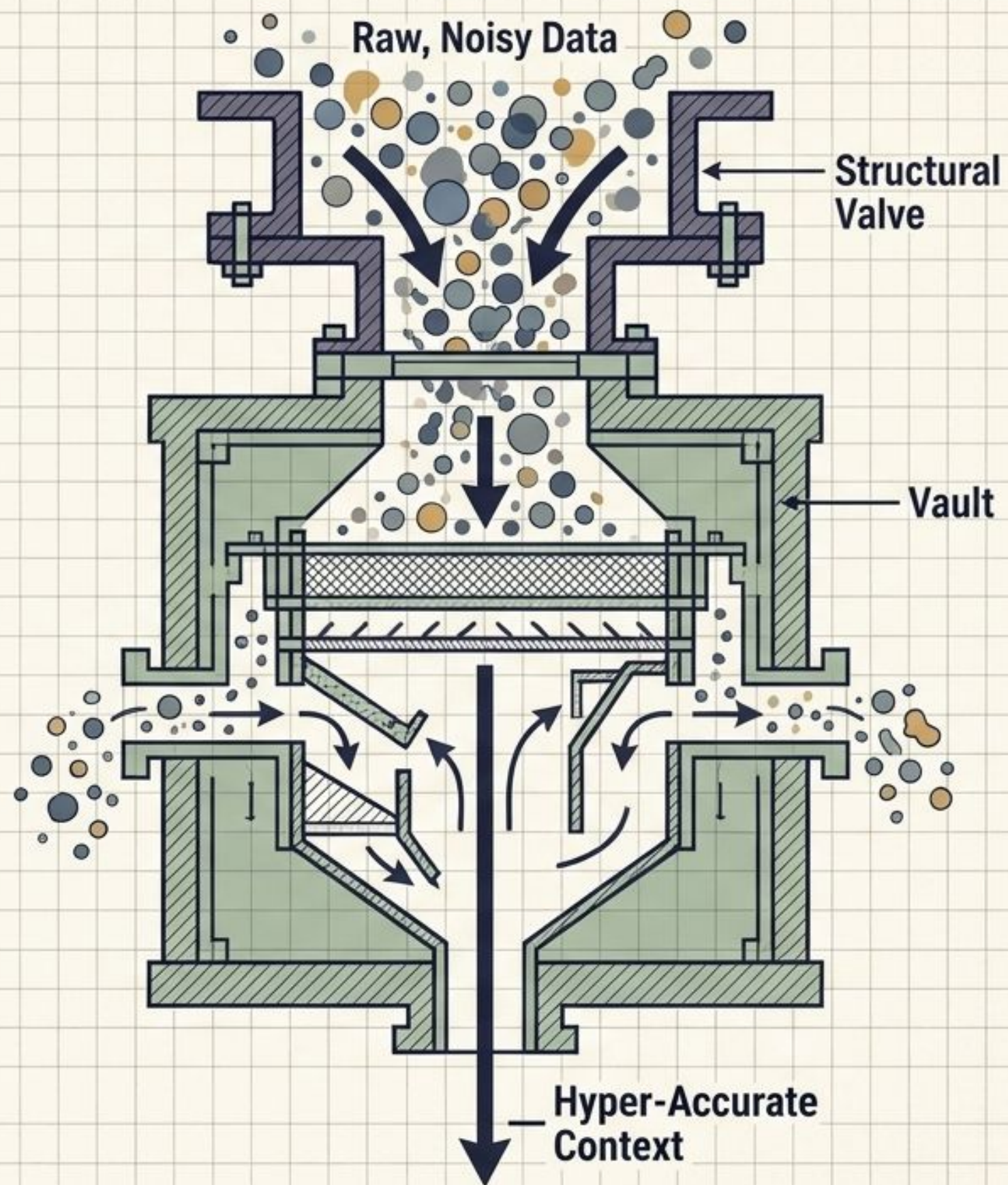
最強の技術スタック：Google Cloud × Elastic

データアクセスからUIレンダリングまで、各機能がシームレスに連携する生きたエコシステム。



【Challenge 1】 検索精度の極大化： ただのRAGからの 脱却

生成AIの回答の質は、与えられるコンテキスト（検索結果）の質に直結する。
Elasticsearchの強力な検索パイプラインを駆使し、ハルシネーションの根絶を図る。



Elasticsearch: 3段階の検索高度化アプローチ

ログ（LTR）と意味理解（Semantic）の双方から「最も確からしい事実」を抽出し、完璧なコンテキストを生成する。

Stage 1: Query Rescore

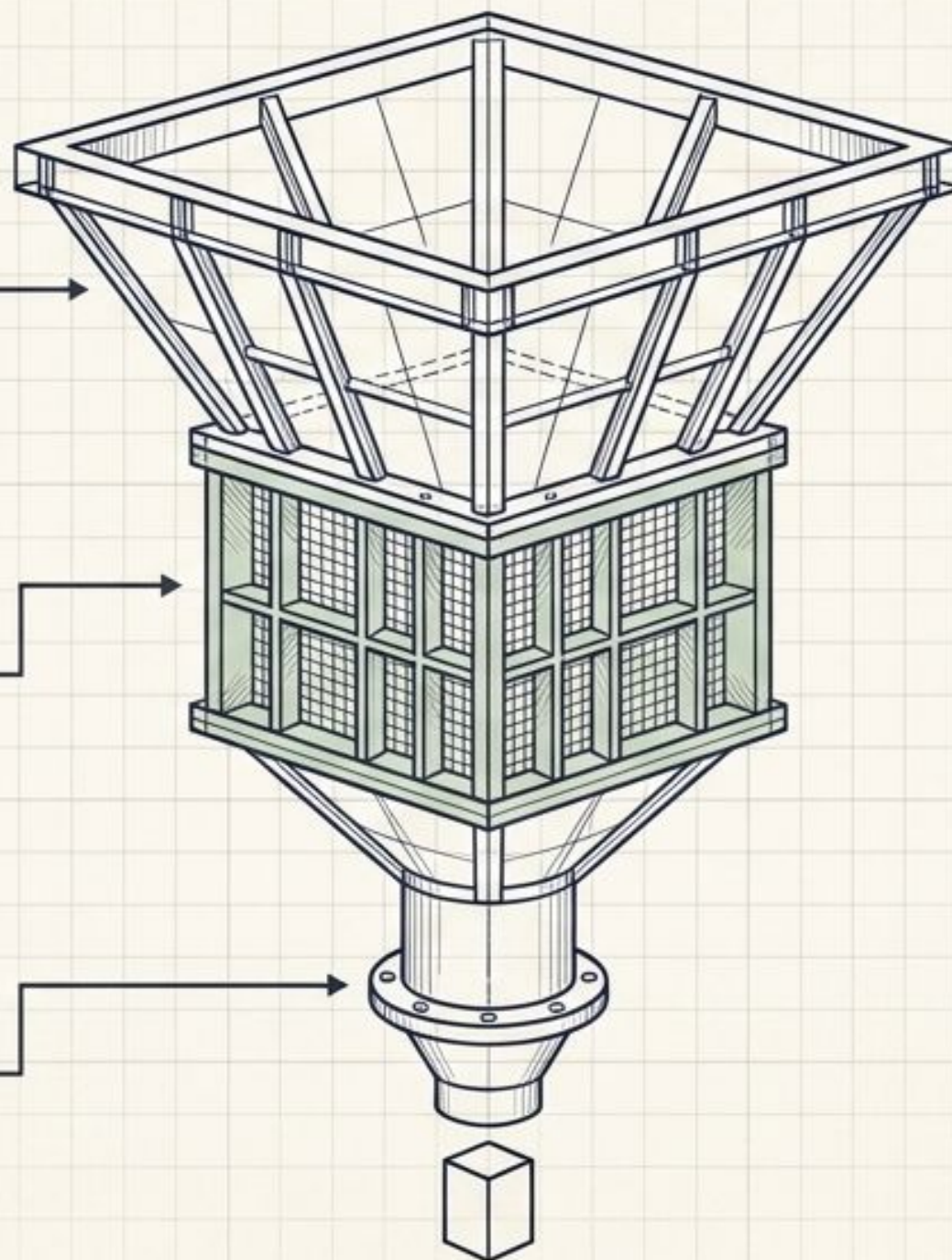
BM25等による高速検索に対し、Painless スクリプト等でビジネスロジックや鮮度を加味して微調整。

Stage 2: Learning to Rank (LTR)

過去のクリックやコンバージョン等のユーザー行動ログを教師データとし、XGBoost等の機械学習モデルで最適化。

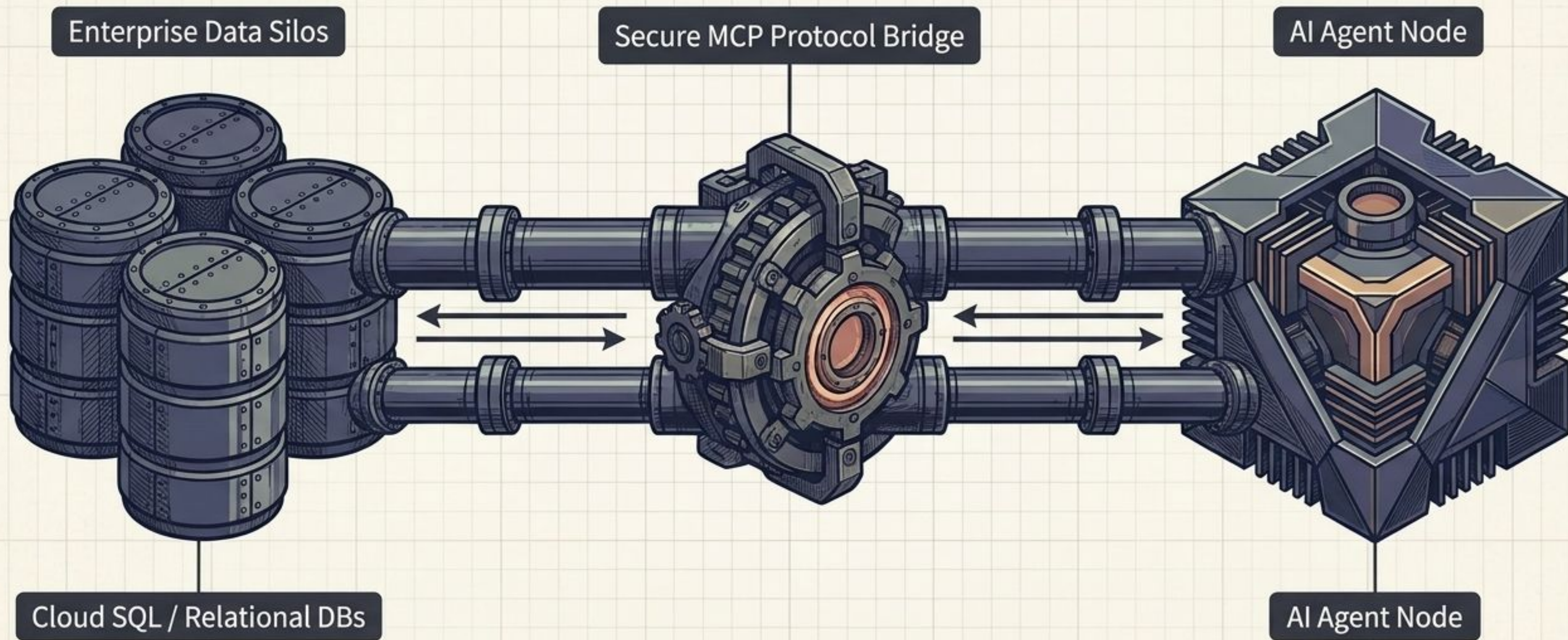
Stage 3: Semantic Reranking

Cohere等のクロスエンコーダーモデル (LLM) を用い、深い意味的つながり（文脈）を理解して最終的な並び替えを実行。



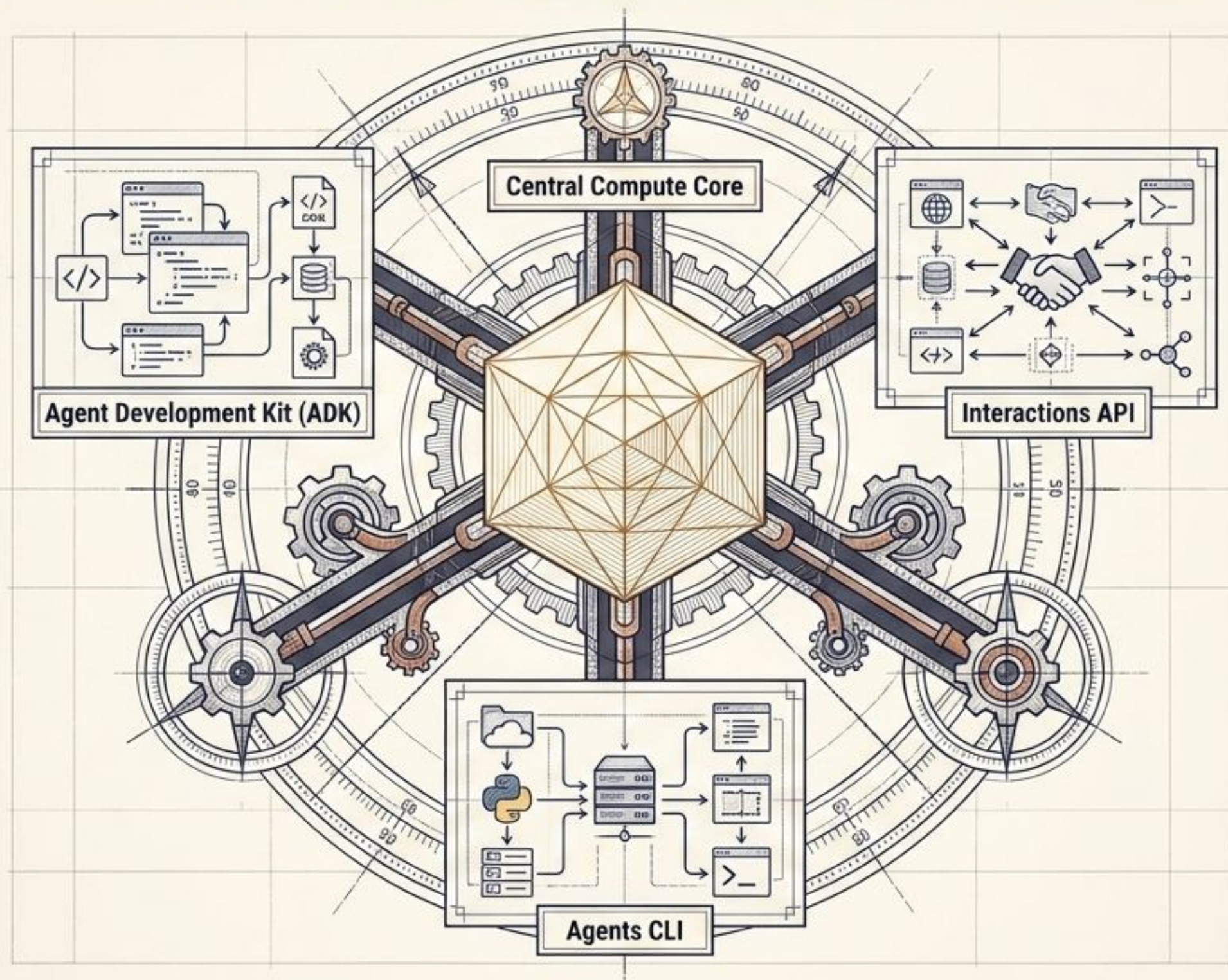
MCP Toolbox: エンタープライズDBとの安全な直結

Model Context Protocol (MCP) と「MCP Toolbox for Databases」により、エージェントが自律的にSQLを発行し、リアルタイムに社内DBへ安全にアクセス可能になる。



【Challenge 2】 エージェントロジックの中核：Google Cloud

複雑なタスクをこなし、自律的なワークフローを管理するための、堅牢で自由度の高いバックエンド開発エコシステム。



爆速開発を実現する最新ツール群

これらのツール群により、インフラ構築の手間を省き、エージェントのコアロジック開発に集中できる。

Interactions API

次世代の標準API

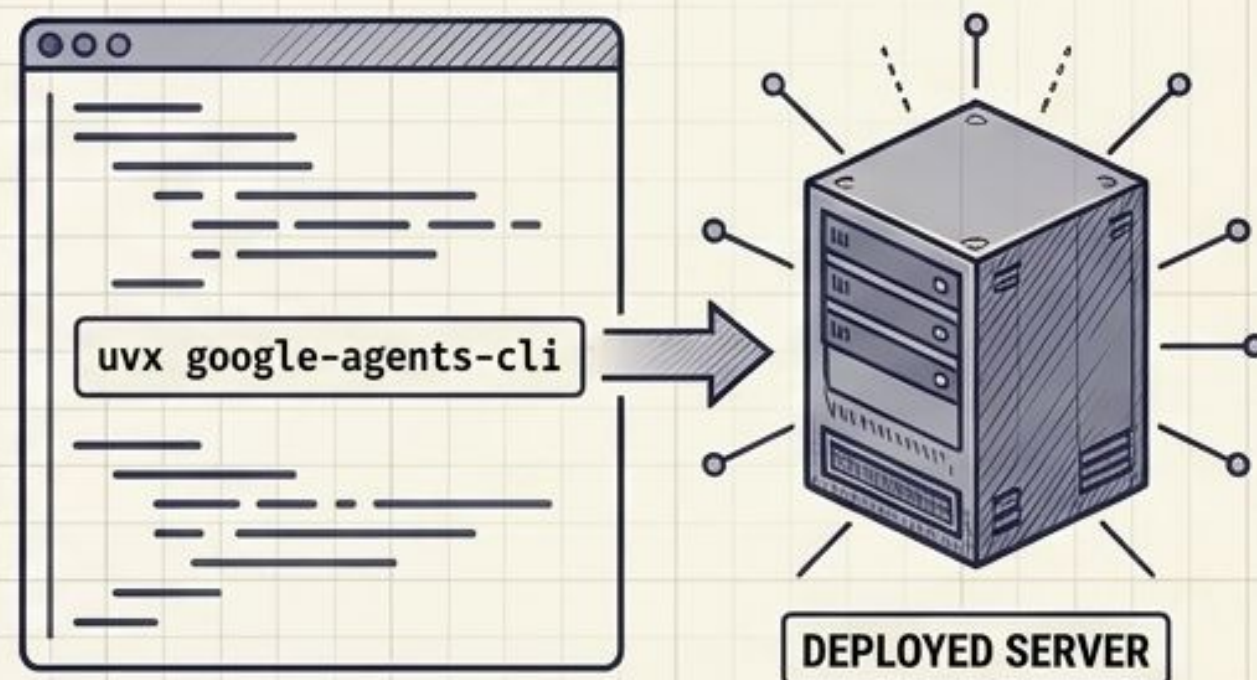
サーバーサイドでの状態管理や、Deep Researchエージェントのような長時間実行される非同期タスクの呼び出し・管理に不可欠な基盤。



Agent CLI & ADK

シームレスなデプロイ

Agent Development Kit (ADK) を用いPythonで記述し、`uvx google-agents-cli` コマンドで一撃デプロイ。Cursor等のAIコーディングツールとの圧倒的なシナジー。



実行環境の選定：Cloud Run vs Agent Runtime

A2UIを用いた圧倒的なリッチUI体験を構築するためには、自由度の高い「Cloud Run」の選択が絶対条件となる。

Feature	Agent Runtime	Cloud Run
手軽さ (Ease of Setup)	High (フルマネージド)	Medium (コンテナベース)
インフラ自由度 (Infra Customization)	Low	High
A2UI等の高度なプロトコル対応 (A2UI Readiness)	Limited	<u>Full Support</u>
推奨ユースケース (Recommended Use Case)	素早いプロトタイピング	<u>リッチUIと複雑な連携を伴う本番環境</u>

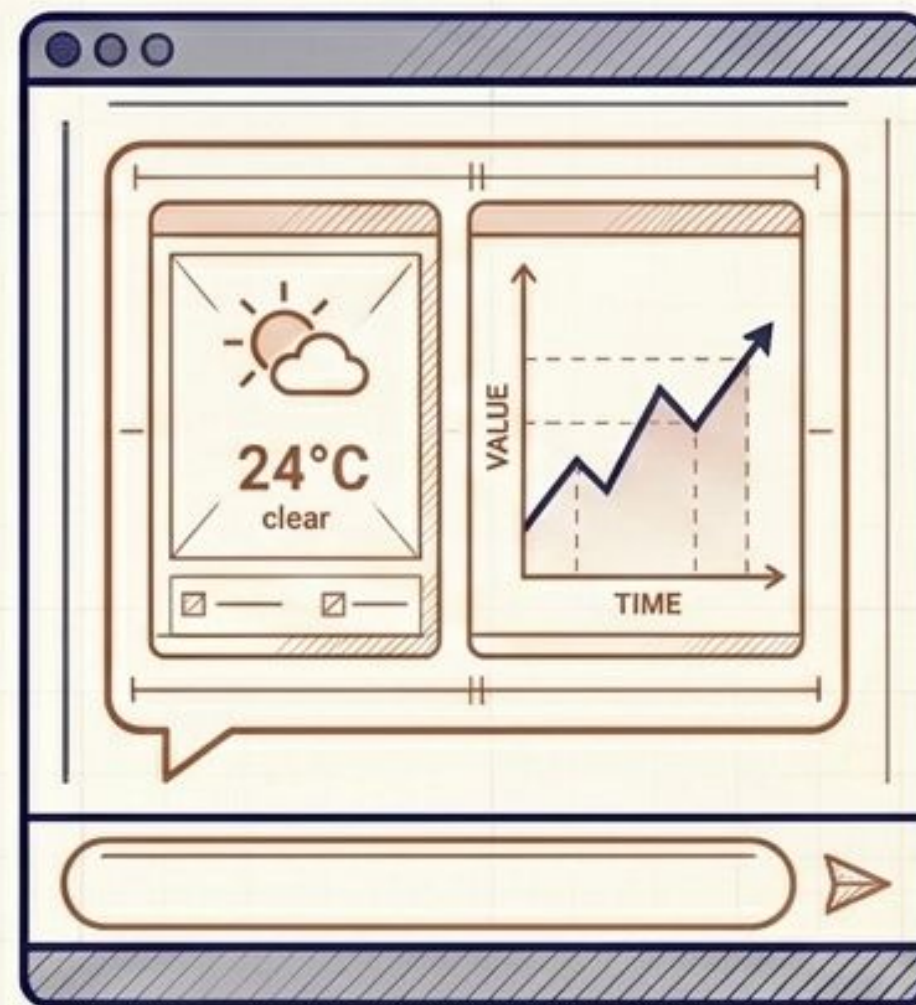
【Challenge 3】テキストから「体験」へ：リッチUIの構築

ユーザーの状況に応じて、最も適したUIコンポーネントをエージェントが「自律的に生成」する次世代のインターフェース。

BEFORE

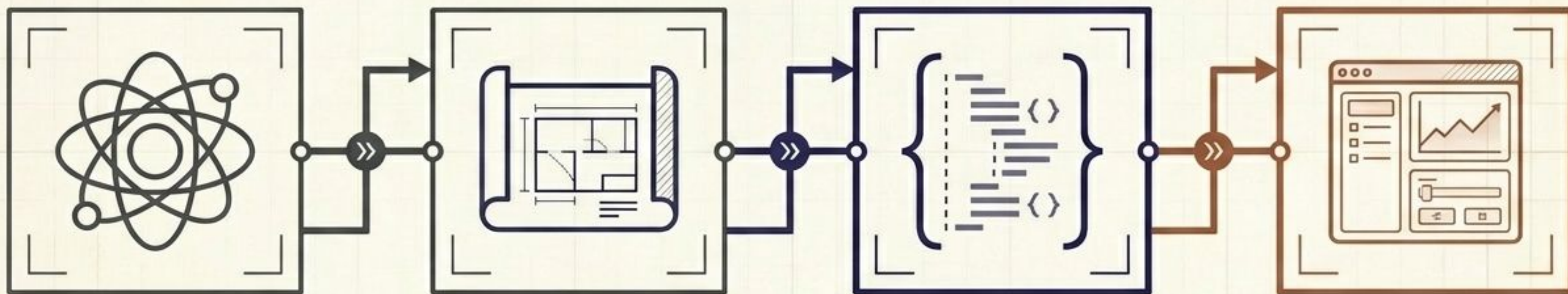


AFTER



A2A / A2UI プロトコルのメカニズム

開発者はフロントエンド実装を意識せず、エージェントのロジックとUIスキーマ定義にのみ集中できる。



1. A2A (相互運用)

Cloud Run上のエージェントがサブエージェントやDBと対話して情報を収集。

2. ADK (スキーマ定義)

エージェントが画面の「構造」を決定。

3. A2UI Payload

テキストではなく「構造化されたJSON」を出力。

4. Renderer

クライアント（Gemini Enterprise等）のレンダラーがJSONを解釈し、ネイティブなUI（カード等）として描画。

全体アーキテクチャ：次世代AIエージェントの青写真

この統合アーキテクチャこそが、ハルシネーションのない「正確なデータ」と「直感的でリッチなUI」を両立させる唯一の解である。

